

Τεχνητή Νοημοσύνη Εργασία 2

Αλέξανδρος Μακρυγιάννης : p3210271
Μάριος Δημήτριος Καπότης: p3220065
Ελισάβετ Μαρία Μπινετοπούλου: p3220132

Μέρος Α – Περιγραφή του Προγράμματος

Σε αυτό το μέρος της εργασίας, αποφασίσαμε να υλοποιήσουμε τρεις αλγορίθμους ταξινόμησης για το σύνολο δεδομένων κριτικών ταινιών (IMDB dataset):

- Αφελής ταξινομητής Bayes
- Τυχαίο Δάσος (Random Forest)
- Λογιστική Παλινδρόμηση

Για την υλοποίηση των παραπάνω αλγορίθμων και για την προεπεξεργασία των δεδομένων, κατασκευάσαμε το σύστημα μας με τις ακόλουθες κύριες κλάσεις:

1. IMDBDataset και IMDBLoader

IMDBDataset

Η κλάση αυτή αποθηκεύει τα ακατέργαστα δεδομένα του IMDB dataset. Συγκεκριμένα, χρησιμοποιεί λίστες τύπου **String** για τα κείμενα των reviews (txt files) των φακέλων **train** και **test**, καθώς και λίστες τύπου **Integer** για τις ετικέτες (labels) – όπου ορίζουμε ότι οι θετικές κριτικές αντιστοιχούν σε 0 και οι αρνητικές σε 1.

IMDBLoader

Η κλάση αυτή αναλαμβάνει την ανάκτηση των δεδομένων. Λαμβάνει ως είσοδο το directory όπου βρίσκεται το IMDB (στην δική μας περίπτωση ήταν “data”) dataset και διαβάζει τα αρχεία κειμένου (txt) από τους φακέλους **train/pos**, **train/neg**, **test/pos** και **test/neg**. Για κάθε αρχείο, αποθηκεύει το περιεχόμενο σε μια λίστα **String** και το αντίστοιχο label (0 για θετικά και 1 για αρνητικά) σε μια ξεχωριστή λίστα. Στο τέλος, επιστρέφει ένα αντικείμενο **IMDBDataset** που περιέχει όλα τα δεδομένα.

2. VocabularyBuilder

Η κλάση **VocabularyBuilder** είναι υπεύθυνη για την εξαγωγή και κατασκευή του λεξιλογίου που θα χρησιμοποιηθεί για την αναπαράσταση των κειμένων ως διανύσματα χαρακτηριστικών.

Η βασική μέθοδος, **buildVocabulary**, λαμβάνει ως είσοδο τα training reviews, τις ετικέτες τους και τις παραμέτρους **n**, **k** και **m**:

- **n**: Ο αριθμός των πιο συχνών λέξεων που απορρίπτονται (καθώς συνήθως είναι πολύ γενικές).
- **k**: Ο αριθμός των πιο σπάνιων λέξεων που απορρίπτονται (λόγω έλλειψης επαναλήψεων).
- **m**: Το μέγεθος του λεξιλογίου που θα επιλεγεί, δηλαδή, οι m λέξεις με το υψηλότερο Information Gain.

Η μέθοδος:

- Υπολογίζει τη συχνότητα εμφάνισης κάθε λέξης (document frequency) στα training κείμενα.
- Ταξινομεί τις λέξεις κατά συχνότητα και απορρίπτει τις n πιο συχνές και k πιο σπάνιες λέξεις.
- Υπολογίζει το Information Gain για κάθε υποψήφια λέξη
- Επιλέγει τις m λέξεις με το υψηλότερο Information Gain και επιστρέφει ένα **Map** όπου τα keys είναι οι επιλεγμένες λέξεις και τα values είναι οι δείκτες (0 έως $m-1$) που θα χρησιμοποιηθούν για την μετατροπή των κειμένων σε διανύσματα.

3. FeatureBuilder

Η κλάση **FeatureBuilder** μετατρέπει τα κείμενα (reviews) σε διανύσματα. Η μέθοδος **buildFeatureMatrix** χρησιμοποιεί το λεξιλόγιο που κατασκευάστηκε από την **VocabularyBuilder** για να δημιουργήσει έναν δισδιάστατο πίνακα, όπου κάθε γραμμή αντιπροσωπεύει ένα κείμενο και έχει m στήλες. Σε κάθε θέση του διανύσματος, αν η αντίστοιχη λέξη του λεξιλογίου εμφανίζεται στο κείμενο, η τιμή τίθεται σε 1 (διαφορετικά παραμένει 0). Η βοηθητική μέθοδος **extractFeatures** πραγματοποιεί την επεξεργασία για κάθε κείμενο ξεχωριστά.

4. DataSplit

Αυτή η κλάση χρησιμοποιείται για την αποθήκευση δύο πινάκων δεδομένων, ενός για το **training set** (εκπαίδευση) και ενός για το **development set** (ανάπτυξη). Ο κατασκευαστής δέχεται δύο δισδιάστατους πίνακες ακεραίων, έναν για το training set και έναν για το development set, και τους αποθηκεύει στα αντίστοιχα πεδία της κλάσης. Περιλαμβάνει:

- Έναν δισδιάστατο πίνακα ακεραίων που αντιπροσωπεύει τα δεδομένα εκπαίδευσης (training set): `public int[][] train;`
- Έναν δισδιάστατο πίνακα ακεραίων που αντιπροσωπεύει τα δεδομένα ανάπτυξης (development set): `public int[][] dev;`

5. DataSplitPair

Αυτή η κλάση χρησιμοποιείται για την αποθήκευση των δεδομένων εκπαίδευσης και ανάπτυξης, αλλά χωρίζει τα δεδομένα σε **features** (χαρακτηριστικά) και **labels** (ετικέτες). Ο κατασκευαστής δέχεται τέσσερις πίνακες: τα χαρακτηριστικά και τις ετικέτες για το **training set** και τα **χαρακτηριστικά** και τις **ετικέτες** για το **development set**, και τα αποθηκεύει στα αντίστοιχα πεδία της κλάσης. Περιλαμβάνει:

- Έναν δισδιάστατο πίνακα ακεραίων που αντιπροσωπεύει τα χαρακτηριστικά του training set: `public int[][] trainFeatures;`
- Έναν μονοδιάστατο πίνακα ακεραίων που αντιπροσωπεύει τις ετικέτες του training set: `public int[] trainLabels;`
- Έναν δισδιάστατο πίνακα ακεραίων που αντιπροσωπεύει τα χαρακτηριστικά του development set: `public int[][] devFeatures;`
- Έναν μονοδιάστατο πίνακα ακεραίων που αντιπροσωπεύει τις ετικέτες του development set: `public int[] devLabels;`

6. DataUtils

Αυτή η κλάση περιέχει μεθόδους βοηθητικές για την επεξεργασία δεδομένων. Η κύρια μέθοδος που υλοποιείται εδώ είναι η **splitFeaturesAndLabels**, η οποία χωρίζει τα δεδομένα σε **training** και **development** σύνολα.

- **public static DataSplitPair splitFeaturesAndLabels(int[][] features, int[] labels, double trainFraction):**
 - *features*: Ένας δισδιάστατος πίνακας ακεραίων που αντιπροσωπεύει τα χαρακτηριστικά όλων των δεδομένων.
 - *labels*: Ένας μονοδιάστατος πίνακας ακεραίων που αντιπροσωπεύει τις ετικέτες όλων των δεδομένων.
 - *trainFraction*: Ένα double που αντιπροσωπεύει το ποσοστό των δεδομένων που θα χρησιμοποιηθούν για το training set.

Αυτή η μέθοδος δημιουργεί μια λίστα με δείκτες για όλα τα δεδομένα. Ανακατεύει τυχαία αυτούς τους δείκτες χρησιμοποιώντας τη μέθοδο *Collections.shuffle*. Υπολογίζει το μέγεθος του training set βάσει του **trainFraction**. Δημιουργεί νέους πίνακες για τα χαρακτηριστικά και τις ετικέτες του training και development set. Γεμίζει τους πίνακες με τα δεδομένα που αντιστοιχούν στους ανακατεμενους δείκτες και επιστρέφει ένα αντικείμενο DataSplitPair που περιέχει τα training και development δεδομένα.

7. Metrics

Αυτή η κλάση χρησιμοποιείται για την αποθήκευση των μετρικών από την αξιολόγηση ενός μοντέλου μηχανικής μάθησης. Περιλαμβάνει μετρικές όπως **precision**, **recall** και **F1-score** για δύο κλάσεις (0 και 1), καθώς και **micro** και **macro** μέσους όρους:

- *precision0, recall0, f10*: precision, recall και F1-score για την κλάση 0.
- *precision1, recall1, f11*: precision, recall και F1-score για την κλάση 1.
- *microPrecision, microRecall, microF1*: Μέσο micro precision, recall και F1-score.
- *macroPrecision, macroRecall, macroF1*: Μέσο macro precision, recall και F1-score.

Η μέθοδος *toString()* επιστρέφει μια αναπαράσταση σε μορφή συμβολοσειράς όλων των μετρικών, με στρογγυλοποίηση σε 3 δεκαδικά ψηφία.

8. MetricsCalculator

Αυτή η κλάση περιέχει μεθόδους για τον υπολογισμό των μετρικών από τις πραγματικές **ετικέτες** (true labels) και τις **προβλεπόμενες ετικέτες** (predicted labels) ενός μοντέλου. (Παίρνει παραμέτρους trueLabels, predictedLabels που είναι οι πίνακες ακεραίων που αντιπροσωπεύουν κάθε είδος ετικέτας). Αυτή η κλάση Υπολογίζει τα true positives (tp), false positives (fp) και false negatives (fn) για κάθε κλάση (0 και 1), τα precision, recall και F1-score για κάθε κλάση, τα micro-average και macro-average precision, recall και F1-score. Τέλος, επιστρέφει ένα αντικείμενο Metrics που περιέχει όλες τις υπολογισμένες μετρικές.

9. Pair

Αυτή η κλάση είναι μια γενική κλάση που χρησιμοποιείται για την αποθήκευση ενός ζεύγους τιμών. Μπορεί να χρησιμοποιηθεί για οποιοδήποτε ζεύγος τύπων δεδομένων. Ο κατασκευαστής δέχεται δύο τιμές και τις αποθηκεύει στα πεδία **first** και **second**.

- **public T first**: Η πρώτη τιμή του ζεύγους.
- **public U second**: Η δεύτερη τιμή του ζεύγους.

Αλγόριθμοι

Τυχαίο Δάσος (Random Forest)

Ο αλγόριθμος Random Forest κατασκευάζει πολλαπλά δέντρα (ένα σύνολο από DecisionTree αντικείμενα), όπου κάθε δέντρο εκπαιδεύεται σε ένα **bootstrap δείγμα** των δεδομένων. Η τελική πρόβλεψη για ένα νέο δείγμα λαμβάνεται μέσω πλειοψηφίας των προβλέψεων που δίνει κάθε ένα δέντρο απόφασης.

Κάθε **δέντρο αποφάσεων** δημιουργείται επιλέγοντας τυχαία ένα υποσύνολο χαρακτηριστικών σε κάθε βήμα και επιλέγοντας το χαρακτηριστικό που μεγιστοποιεί το κέρδος πληροφορίας για το split των δεδομένων. Με αυτόν τον τρόπο, δημιουργούνται δυαδικά δέντρα όπου κάθε εσωτερικός κόμβος αντιπροσωπεύει μια απόφαση και κάθε φύλλο αποδίδει μια πρόβλεψη (0 ή 1).

Χρήση του Προγράμματος

Για τη χρήση του προγράμματός μας, ο χρήστης:

- Ορίζει το directory του IMDB dataset.
- Εκτελεί το πρόγραμμα, το οποίο ανακτά και επεξεργάζεται τα δεδομένα, κατασκευάζει το λεξιλόγιο και μετατρέπει τα κείμενα σε διανύσματα.
- Τρέχει τους αλγορίθμους ταξινόμησης με τις επιλεγμένες υπερπαραμέτρους.
- Παρακολουθεί τα learning curves και λαμβάνει τους τελικούς πίνακες αποτελεσμάτων για την αξιολόγηση της απόδοσης στα training, development και test δεδομένα.

Λογιστική Παλινδρόμηση (Logistic Regression)

Η κλάση Logistic Regression υλοποιεί τον αλγόριθμο λογιστικής παλινδρόμησης, ο οποίος χρησιμοποιείται για δυαδική ταξινόμηση. Ο αλγόριθμος εκπαιδεύεται μέσω βαθμίδωσης καθόδου (gradient descent) και χρησιμοποιεί τη σιγμοειδή (sigmoid) συνάρτηση ενεργοποίησης. Η κλάση αυτή περιλαμβάνει:

- Τον πίνακα βαρών του μοντέλου που περιλαμβάνει και το bias: *private double[] weights*
- Τον ρυθμό της μάθησης: *private double learningRate*
- Συντελεστής κανονικοποίησης (λ) για αποφυγή overfitting: *private double lambda*
- Τον αριθμό επαναλήψεων της εκπαίδευσης: *private int epochs*

Οι μέθοδοι της Logistic Regression είναι οι εξής:

- **sigmoid(double z):** Υπολογίζει την σιγμοειδή συνάρτηση $\sigma(z) = 1 / (1 + e^{(-z)})$. Χρησιμοποιείται για την πρόβλεψη πιθανότητας μιας κλάσης.
- **train(double[][] x, int[] y):** Εκτελεί *epochs* αριθμό επαναλήψεων gradient descent. Σε κάθε επανάληψη υπολογίζει τις προβλέψεις ($h = \text{sigmoid}(X * \text{weights})$), υπολογίζει το λάθος (error) $h - y$, προσαρμόζει τα βάρη με την συνάρτηση $\text{weights}[j] -= \text{learningRate} * (\text{sum}(\text{errors} * X_j) + \text{lambda} * \text{weights}[j])$; (η κανονικοποίηση $\text{lambda} * \text{weights}[j]$ αποτρέπει το overfitting).
- **predict(double[] x):** Υπολογίζει $\sigma(X * \text{weights})$ και επιστρέφει 1 αν η πιθανότητα > 0.5 , αλλιώς 0.

Καμπύλες μάθησης και πίνακες

Για τη δημιουργία του λεξιλογίου χρησιμοποιούμε τις παραμέτρους:

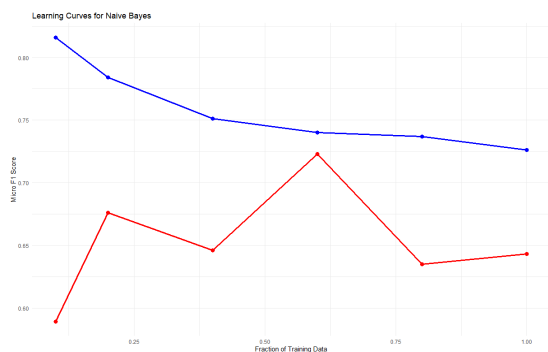
- **n = 75**
- **k = 1250**
- **m = 800**

Κατά την εκπαίδευση, όλα τα μοντέλα χρησιμοποιούν το ίδιο λεξιλόγιο. Για την κατασκευή των καμπυλών μάθησης, επιλέξαμε ως δείγμα κάποια αποτελέσματα που παρήγαγε ο κώδικας και μετατρέψαμε τα δεδομένα σε γραφήματα με τη χρήση της γλώσσας R.

Naive Bayes

Για τον αλγόριθμο του Naive Bayes χρησιμοποιήσαμε ως alpha 0.1 γιατί τα δεδομένα που δίνουμε για εκπαίδευση είναι πολλά, επομένως θέλουμε μικρό alpha.

Καμπύλη μάθησης:



Πίνακας:

	Precision	Recall	F1
Class 0	0.931	0.918	0.925
Class 1	0.916	0.929	0.922
Micro Avg	0.924	0.924	0.924
Macro Avg	0.924	0.924	0.924

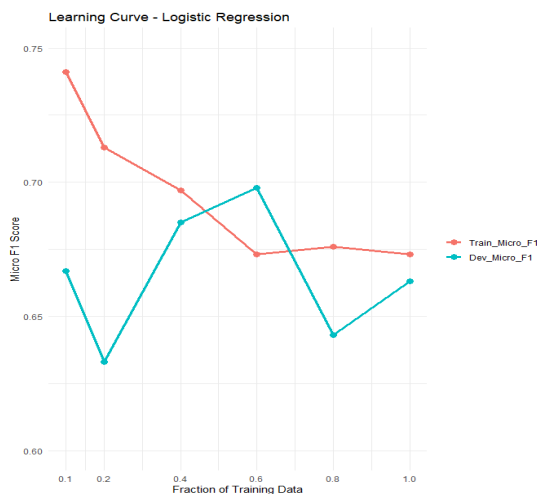
Τα αποτελέσματα αυτά δείχνουν ότι:

- 1)Το μοντέλο έχει πολύ καλή συνολική απόδοση (F1: 0.924)
- 2)Ισορροπημένη ακρίβεια (Precision) και ανάκληση (Recall) για αμφότερες τις κλάσεις
- 3)Χαμηλό overfitting, καθώς η διαφορά Train & Dev F1 είναι μικρότερη από ό,τι στο Random Forest.
- 4)Το Dev F1 δεν αυξάνεται σημαντικά με περισσότερα δεδομένα.
- 5) Χαμηλότερη γενίκευση από άλλα μοντέλα, καθώς το validation set δεν ξεπερνά το 0.723.

Λογιστική Παλινδρόμηση

Για τον αλγόριθμο της Λογιστικής Παλινδρόμησης χρησιμοποιούμε τις εξής υπερπαραμέτρους:

- Αριθμός χαρακτηριστικών (numFeatures): m
- Ρυθμός εκμάθησης (learningRate): 0.1
- Συντελεστής κανονικοποίησης (lambda): 0.01
- Εποχές (epochs): 1000



	Precision	Recall	F1
Class 0	0.934	0.874	0.903
Class 1	0.855	0.924	0.888
Micro Avg	0.896	0.896	0.896
Macro Avg	0.894	0.899	0.895

Τα αποτελέσματα αυτά δείχνουν ότι:

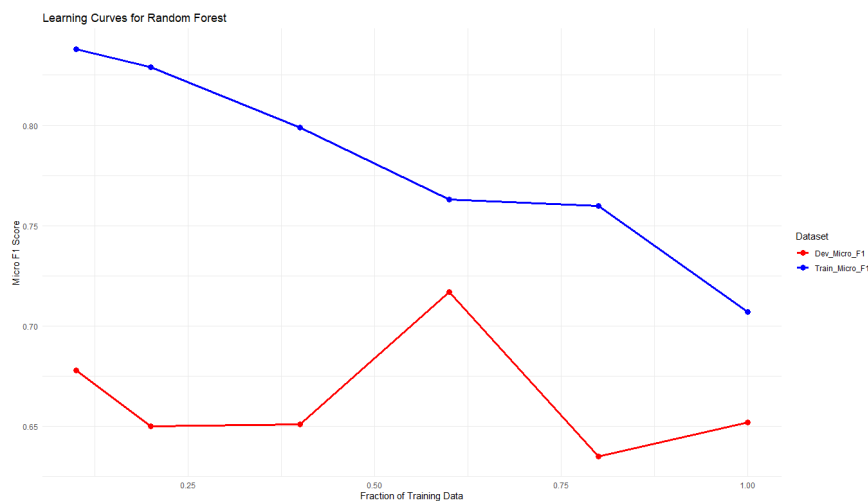
- 1)Καλή ισορροπία μεταξύ Precision και Recall.
- 2) Δεν φαίνεται να υπερεκπαιδεύεται, κάτι που είναι θετικό για μελλοντική χρήση σε νέα δεδομένα.
- 3) το μοντέλο φαίνεται να έχει αστάθεια στη γενίκευση.

Τυχαίο Δάσος

Για τον αλγόριθμο του Random Forest χρησιμοποιήσαμε τις εξής υπερπαραμέτρους:

- Αριθμός δέντρων απόφασης: 1000
- Μέγιστο βάθος κάθε δέντρου: 10
- Ελάχιστος αριθμός δειγμάτων για να πραγματοποιηθεί το split: 5
- Ο αριθμός των χαρακτηριστικών που θα εξεταστούν τυχαία σε κάθε κόμβο ενός δέντρου (για την επιλογή της καλύτερης διαίρεσης): Το ακέραιο μέρος της τετραγωνικής ρίζας του m

Καμπύλη μάθησης:



Πίνακας:

	Precision	Recall	F1
Class 0	0.972	0.816	0.887
Class 1	0.728	0.955	0.826
Micro Avg	0.863	0.863	0.863
Macro Avg	0.850	0.885	0.857

Τα

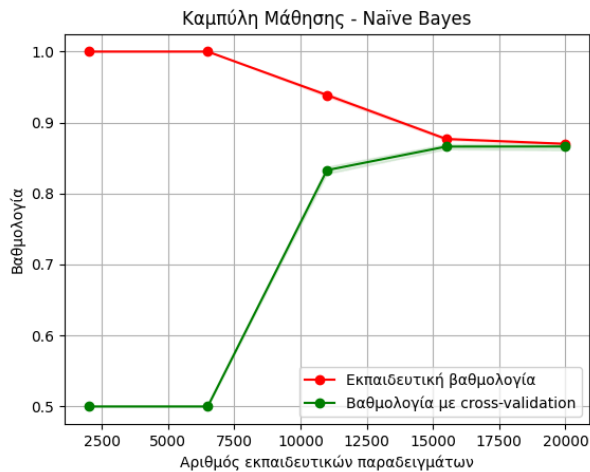
αποτελέσματα αυτά δείχνουν ότι:

- 1) Το μοντέλο, σύμφωνα με τον δείκτη recall στη Class 1 αναγνωρίζει πιο εύκολα τις θετικές κριτικές αλλά κάνει περισσότερα λάθη όταν τις χαρακτηρίζει.
- 2) Είναι πιο ακριβές στις αρνητικές κριτικές (υψηλό Precision στη Class 0), αλλά μερικές φορές τις αγνοεί.
- 3) Τα Train και Dev F1 δείχνουν ότι υπάρχει μικρό overfitting, καθώς η απόδοση στο training set είναι υψηλότερη από το validation set.

Μέρος Β - Σύγκριση Επιδόσεων

Παρακάτω παρατίθεται ένα από τα αποτελέσματα που έβγαλε το Scikit-learn και για τους 3 αλγόριθμους

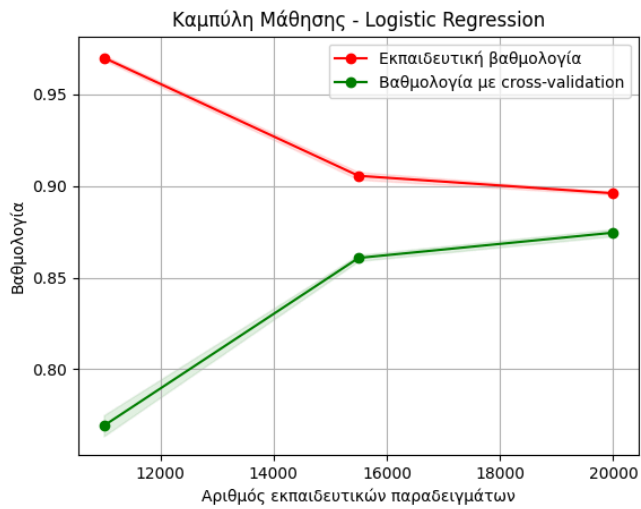
Naive Bayes



	Precision	Recall	F1
Class 0	0.85	0.75	0.80
Class 1	0.78	0.87	0.82
Micro Avg	0.81	0.81	0.81
Macro Avg	0.81	0.81	0.81

Η δικιά μας υλοποίηση του Naïve Bayes έχει καλύτερα αποτελέσματα από αυτόν σε όλες τις μετρήσεις. Αυτό σημαίνει ότι το μοντέλο μας είναι πιο αποδοτικό και μπορεί να κάνει καλύτερη ταξινόμηση των δεδομένων.

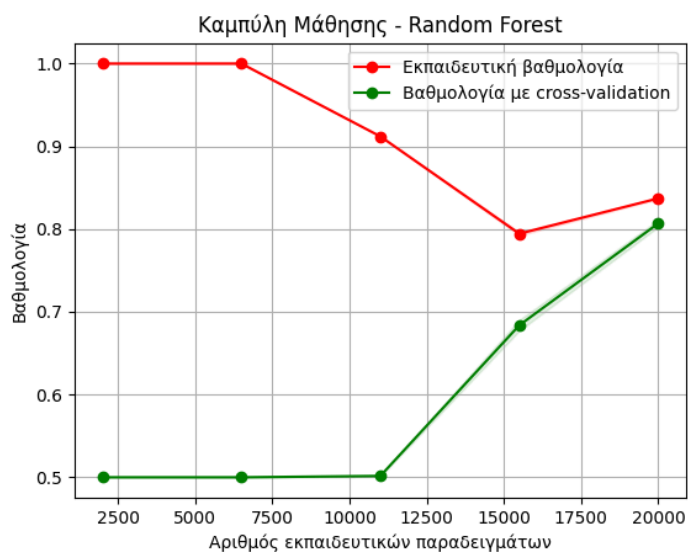
Λογιστική Παλινδρόμηση



	Precision	Recall	F1
Class 0	0.88	0.75	0.87
Class 1	0.87	0.88	0.87
Micro Avg	0.87	0.87	0.87
Macro Avg	0.87	0.87	0.87

Ο δικός μας ο Logistic Regression είναι καλύτερος διότι έχει υψηλότερο Accuracy, Precision, Recall και F1-score.

Τυχαίο Δάσος



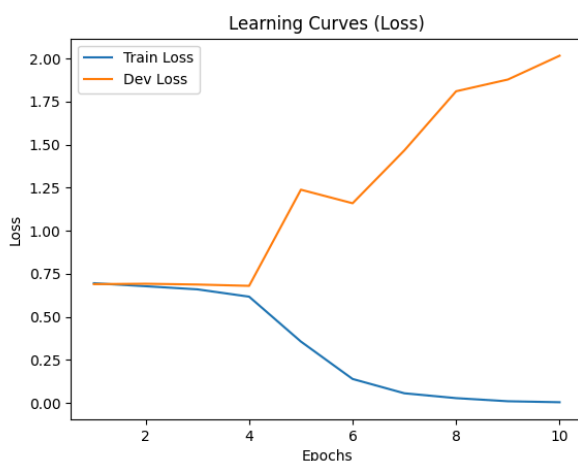
	Precision	Recall	F1
Class 0	0.85	0.75	0.80
Class 1	0.78	0.87	0.82
Micro Avg	0.81	0.81	0.81
Macro Avg	0.81	0.81	0.81

Ο δικός μας αλγόριθμος φαίνεται να αποδίδει συνολικά καλύτερα, με υψηλότερα F1 scores και μικρότερη απόκλιση μεταξύ training και validation performance. Αυτός ο αλγόριθμος έχει πιο ισορροπημένη ακρίβεια μεταξύ των δύο κλάσεων, αλλά φαίνεται να ξεκινά με ισχυρό overfitting και να βελτιώνεται σταδιακά.

Μέρος Γ - Σύγκριση Επιδόσεων

Οι τιμές των υπερπαραμέτρων που χρησιμοποιήσαμε στον κώδικα του PyTorch μοντέλου είναι η ακόλουθες:

- το μέγεθος του λεξιλογίου(vocab_size): 10.000
- διάσταση των word embeddings (embed_dim): 300
- μέγεθος του κρυφού state του RNN(hidden_dim): 128
- αριθμός στοιβαγμένων επιπέδων του RNN (num_layers): 2
- dropout που εφαρμόζεται μεταξύ των επιπέδων (dropout): 0.5
- use_gru (δηλαδή χρησιμοποιείται LSTM αντί για GRU): False
- num_classes (δύο κατηγορίες, π.χ. θετικές και αρνητικές κριτικές): 2
- Optimizer (Adam): learning rate = 0.001
- αριθμός εποχών εκπαίδευσης (num_epochs): 10
- μέγεθος του batch για τα DataLoader (batch_size): 32



	Class 0	Class 1	Micro Avg	Macro Avg
Precision	0.273	0.448	0.390	0.360
Recall	0.196	0.556	0.390	0.376
F1-score	0.228	0.496	0.390	0.362

Συγκρίσεις με το Μέρος A:

- **Naive Bayes**
 - Ο Naive Bayes έχει πολύ καλές επιδόσεις, με F1 scores κοντά στο 0.925 για την κατηγορία 0 και 0.922 για την κατηγορία 1. Οι μέσοι όροι micro και macro είναι επίσης πολύ υψηλοί, περίπου 0.924.
 - Το RNN έχει πολύ χαμηλότερες επιδόσεις, με F1 scores 0.228 για την κατηγορία 0 και 0.496 για την κατηγορία 1. Οι μέσοι όροι είναι επίσης πολύ χαμηλότεροι (0.390). Αυτό δείχνει ότι το Naive Bayes έχει πολύ καλύτερη γενίκευση και απόδοση σε σύγκριση με το RNN.
- **Logistic Regression**
 - Η Logistic Regression έχει επίσης καλές επιδόσεις, με F1 scores 0.903 για την κατηγορία 0 και 0.888 για την κατηγορία 1. Οι μέσοι όροι είναι περίπου 0.896.
 - Όπως και με τον Naive Bayes, η Logistic Regression υπερτερεί σημαντικά του RNN. Το RNN έχει πολύ χαμηλότερες επιδόσεις, γεγονός που υποδηλώνει ότι η Logistic Regression είναι πιο αποτελεσματική για αυτό το πρόβλημα.
- **Random Forest**
 - Το Random Forest έχει F1 scores 0.887 για την κατηγορία 0 και 0.826 για την κατηγορία 1. Οι μέσοι όροι είναι περίπου 0.863.
 - Το Random Forest έχει επίσης καλύτερες επιδόσεις από το RNN, αν και οι διαφορές δεν είναι τόσο μεγάλες όπως με τον Naive Bayes και τη Logistic Regression. Το RNN έχει πολύ χαμηλότερες επιδόσεις, γεγονός που υποδηλώνει ότι το Random Forest είναι πιο αποτελεσματικό.

Συγκρίσεις με το Μέρος B:

- **Naive Bayes**
 - Ο Naive Bayes έχει F1 scores 0.80 για την κατηγορία 0 και 0.82 για την κατηγορία 1. Οι μέσοι όροι micro και macro είναι περίπου 0.81.
 - Το RNN έχει πολύ χαμηλότερες επιδόσεις, με F1 scores 0.228 για την κατηγορία 0 και 0.496 για την κατηγορία 1. Οι μέσοι όροι είναι επίσης πολύ χαμηλότεροι (0.390). Αυτό δείχνει ότι ο Naive Bayes έχει πολύ καλύτερη γενίκευση και απόδοση σε σύγκριση με το RNN.
- **Logistic Regression**
 - Η Logistic Regression έχει F1 scores 0.87 για την κατηγορία 0 και 0.87 για την κατηγορία 1. Οι μέσοι όροι είναι περίπου 0.87.

- Η Logistic Regression υπερτερεί σημαντικά του RNN. Το RNN έχει πολύ χαμηλότερες επιδόσεις, γεγονός που υποδηλώνει ότι η Logistic Regression είναι πιο αποτελεσματική για αυτό το πρόβλημα.
- **Random Forest**
 - Το Random Forest έχει F1 scores 0.80 για την κατηγορία 0 και 0.82 για την κατηγορία 1. Οι μέσοι όροι είναι περίπου 0.81.
 - Το Random Forest έχει επίσης καλύτερες επιδόσεις από το RNN, αν και οι διαφορές δεν είναι τόσο μεγάλες όπως με τη Logistic Regression. Το RNN έχει πολύ χαμηλότερες επιδόσεις, γεγονός που υποδηλώνει ότι το Random Forest είναι πιο αποτελεσματικό.

Συγκρίνοντας τα αποτελέσματα του Μέρους Γ με τα αποτελέσματα των Μερών Α και Β, παρατηρούμε ότι τα κλασικά μοντέλα μηχανικής μάθησης (Naive Bayes, Logistic Regression, Random Forest) έχουν σημαντικά καλύτερες επιδόσεις σε όλες τις μετρικές (precision, recall, F1) σε σύγκριση με το RNN. Αυτό οφείλεται σε μεγάλο βαθμό στο γεγονός ότι το RNN φαίνεται να υπερεκπαιδεύεται, καθώς το train loss συνεχίζει να μειώνεται, ενώ το dev loss αυξάνεται δραματικά μετά τις πρώτες εποχές. Τα κλασικά μοντέλα είναι πιο απλά και λιγότερο επιρρεπή σε υπερεκπαίδευση, γεγονός που τα καθιστά πιο αποτελεσματικά για αυτό το πρόβλημα.

Σημείωση

Τις τιμές των υπερπαραμέτρων για όλα τα μέρη του προγράμματος τις λάβαμε από τα δοκιμαστικά runs που πραγματοποιήσαμε στον κώδικα με διάφορες άλλες τιμές.